



**Sistema de diseño para la interfaz gráfica de aplicaciones móviles asociada a proyectos de
investigación de la Facultad de Ingeniería de Sistemas de la UAN**

Manual Técnico

Gisel Daniela Caicedo Soler

13572229778

Universidad Antonio Nariño

Programa Ingeniería de Sistemas y Computación

Facultad de Ingeniería de Sistemas

Bogotá, Colombia

2026



Contenidos

Introducción	2
1. Descripción general del sistema	3
2. Características de los usuarios del sistema	6
3. Requisitos de hardware y de software	7
4. Instrucciones de instalación	8
5. Solución de problemas	9

Introducción

El presente manual técnico tiene como objetivo proporcionar al personal especializado, desarrolladores y colaboradores de los proyectos de investigación de la Facultad de Ingeniería de Sistemas de la Universidad Antonio Nariño, la documentación exhaustiva y detallada necesaria para la correcta integración, configuración, administración y resolución de incidencias del sistema Uan UI Kit v1.0.0.

El Uan UI Kit es una librería de componentes de interfaz de usuario para aplicaciones Android, implementada en Kotlin con Jetpack Compose. Surge como respuesta a la falta de uniformidad en el diseño de las aplicaciones móviles de los proyectos de investigación de la facultad —como REDSCATE y MULTISALTO—, y tiene como propósito centralizar los criterios visuales, de interacción y de accesibilidad bajo los estándares WCAG 2.2 nivel AA.

1. Descripción General del Sistema

1.1 Nombre y versión del sistema

Nombre del sistema	Uan UI Kit
Versión	v1.0.0
Plataforma objetivo	Android (SDK mínimo 26, SDK objetivo 35)
Lenguaje principal	Kotlin con Jetpack Compose
Tipo de artefacto	Módulo Gradle (.aar)
Repositorio	GitHub Packages — Facultad Ing. Sistemas UAN
Año de desarrollo	2026

1.2 Tecnologías utilizadas

Nombre oficial: Uan UI Kit

Versión actual: 1.0.0

Identificador de artefacto: co.edu.uan.ingsis:ui-kit:1.0.0

Repositorio: github.com/GiselCaicedo/uan-ui-kit

Tipo: Librería Android (.aar) publicada en GitHub Packages

Nota: A lo largo de este documento el sistema se denomina exclusivamente «Uan UI Kit». Las expresiones uan-design-system, uan-ui-kit y ui-kit hacen referencia al mismo producto en

contextos técnicos (nombre del repositorio, módulo Gradle y artefacto respectivamente). Cuando se cite la dependencia en código siempre use: `co.edu.uan.ingsis:ui-kit:1.0.0`

- Kotlin: Lenguaje principal del módulo. Se siguen las convenciones de nomenclatura estándar de Kotlin.
- Jetpack Compose (BOM 2024.09.00): Framework de UI declarativa de Google utilizado para la implementación de todos los componentes.
- Material3: Sistema de diseño de Google, utilizado como base para los tokens semánticos y componentes de superficie.
- Gradle (Kotlin DSL): Sistema de construcción del módulo, con configuración en `build.gradle.kts`.
- GitHub Packages: Registro de paquetes donde se publica el artefacto `.aar` del módulo `:ui-kit`.
- Android TalkBack: Servicio de accesibilidad de Android utilizado para las pruebas de lector de pantalla.

1.3 Arquitectura del sistema

El Uan UI Kit se organiza en un único módulo Gradle denominado `:ui-kit`, independiente del módulo de aplicación que lo consume. Esta separación garantiza que la librería no tiene acceso al código de negocio de la aplicación y puede distribuirse de forma autónoma hacia otros proyectos.

El código se organiza internamente en tres capas:

Capa	Descripción
theme	Contiene el sistema de tokens (UanTokens) y el composable UanTheme, que inyecta los tokens en el árbol de composición mediante CompositionLocalProvider.
foundation	Agrupar las primitivas internas compartidas: el enum unificado UanTone, la primitiva UanIconButton y el objeto UanInteractiveDefaults con constantes de interacción centralizadas.
components	Contiene los dieciséis composables públicos del sistema, cada uno con su API pública documentada.

1.4 Sistema de tokens

El núcleo de la arquitectura visual es la clase de datos UanTokens, que agrupa cuatro subsistemas:

- UanColors: Valores de color (primario, secundario, semánticos: success, warning, danger, critical, info).
- UanTypography: Estilos tipográficos del sistema.
- UanShapes: Radios de borde para los componentes.
- UanMotion: Duraciones de animación y soporte para reduceMotion.

El valor predeterminado UanTokens.Default corresponde al tema oscuro definido por la Facultad de Diseño Gráfico. El módulo consumidor puede personalizar cualquier subconjunto de tokens mediante la función copy() de Kotlin.

1.5 Módulos principales

A continuación se presenta el catálogo de componentes del sistema Uan UI Kit v1.0.0, con la representación visual de cada componente seguida de su implementación en código Kotlin con Jetpack Compose.

UanActivationAnimation

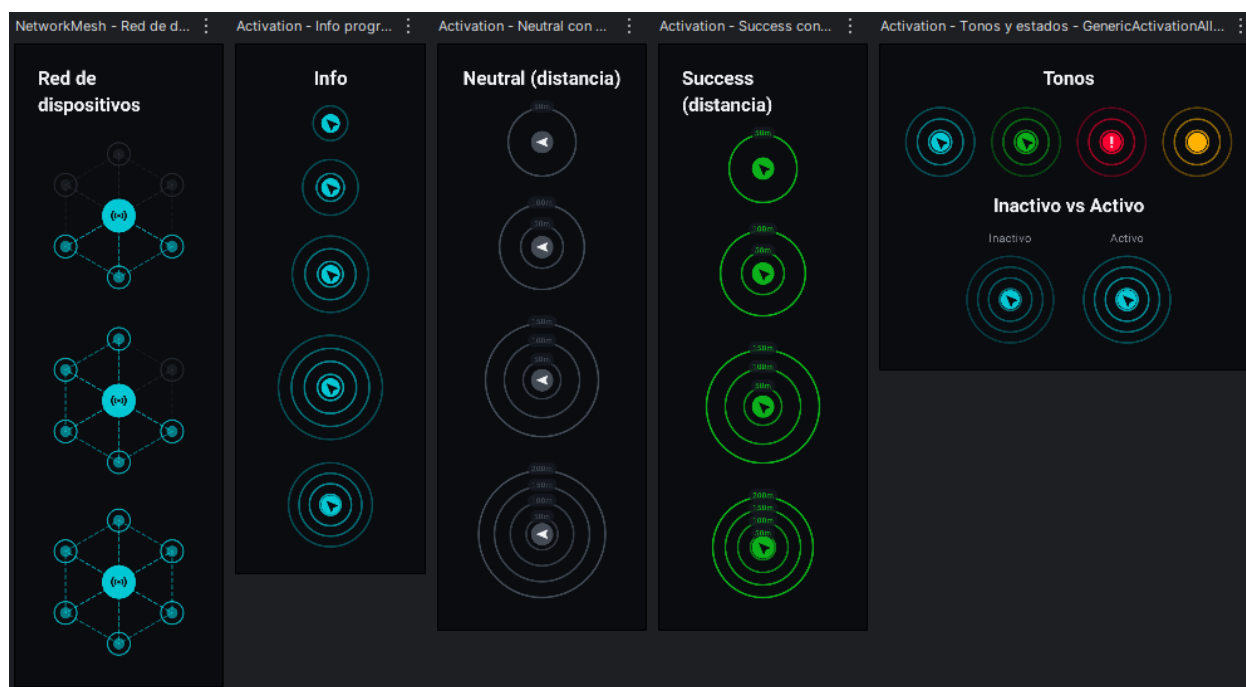


Figura 7. Animación de activación activa

UanActivationAnimation muestra una animación visual que refuerza la activación de un evento o estado interactivo. Se controla mediante el enum UanActivationState.

```
UanActivationAnimation(  
    active = true,  
    tone = UanTone.Primary,  
)
```

UanAppBar

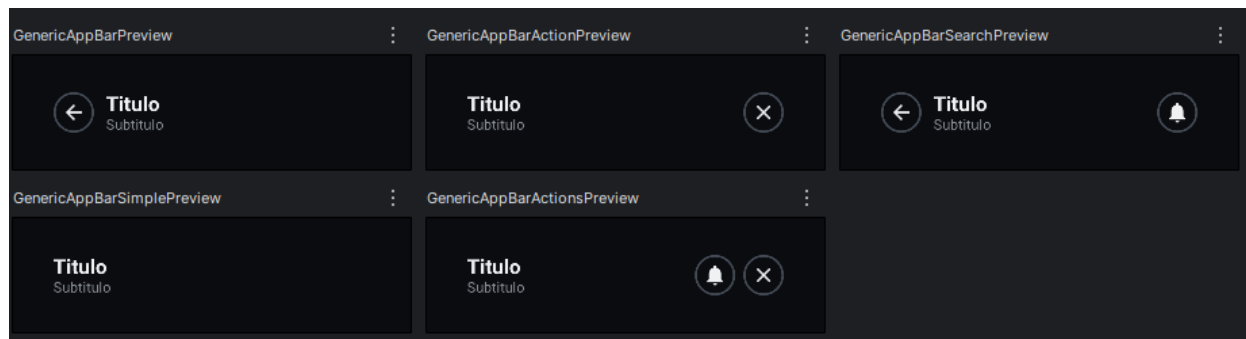


Figura 9. Barra superior de navegación (UanAppBar)

UanAppBar organiza el título, subtítulo, ícono de navegación y las acciones en la parte superior de cada pantalla. Se configura mediante un objeto UanAppBarConfig.

```
UanAppBar(
  title = "Gestión de Alertas",
  subtitle = "Campus Norte",
  navigationIcon = {
    IconButton(onClick = { navController.popBackStack() }) {
      Icon(Icons.Default.ArrowBack, contentDescription = "Volver")
    }
  },
  actions = {
    IconButton(onClick = { }) {
      Icon(Icons.Default.MoreVert, contentDescription = "Más opciones")
    }
  },
)
```

UanBadge

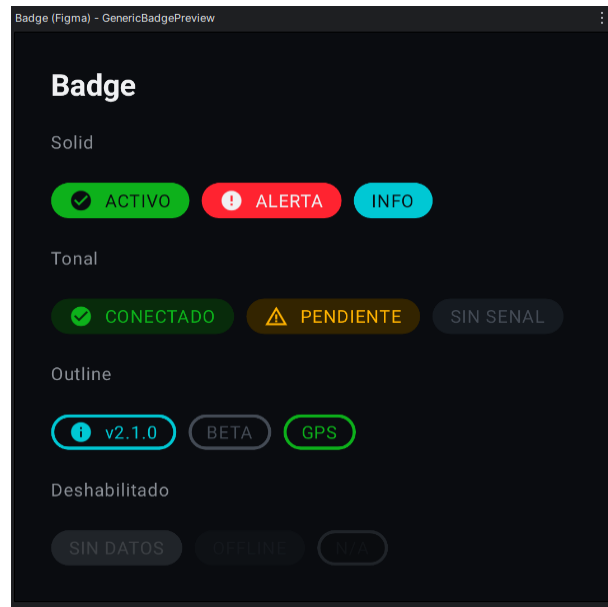


Figura 10. Indicador compacto de estado (UanBadge)

UanBadge muestra contadores, estados o etiquetas semánticas. Soporta cuatro estilos (UanTone) y el parámetro maxCount para limitar la cifra mostrada.

```
UanBadge(
  text = "Activo",
  tone = UanTone.Success,
  emphasis = UanBadgeEmphasis.Tonal,
  leadingContent = {
    Icon(Icons.Default.Circle, contentDescription = null)
  },
)
```

UanIconButton

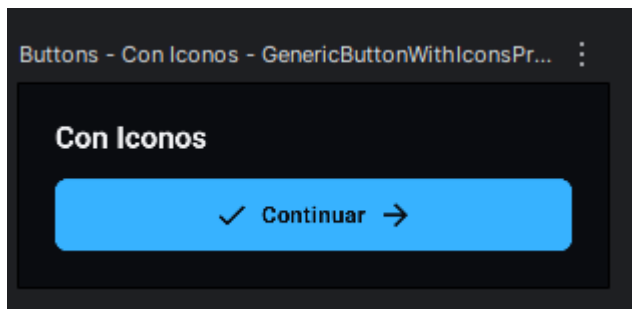


Figura 11. Estilos del botón de ícono (UanIconButton)

UanIconButton es una primitiva de foundation reutilizable para acciones rápidas representadas por un ícono. El parámetro `contentDescription` es obligatorio para garantizar accesibilidad.

```
UanIconButton(  
  icon = Icons.Default.Notifications,  
  contentDescription = "Notificaciones",  
  tone = UanTone.Primary,  
  onClick = { },  
)
```

UanButton

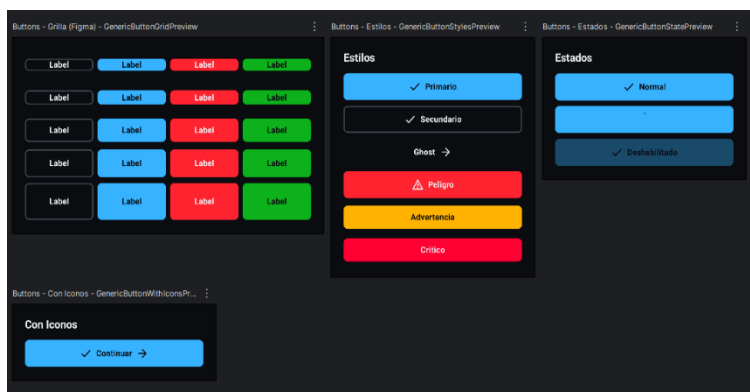


Figura 12. Variantes del botón principal (UanButton)

UanButton es el componente para ejecutar acciones del usuario. Soporta seis estilos visuales, tres tamaños y estado de carga.

```
UanButton(
  text = "Confirmar",
  onClick = { /* acción */ },
  style = UanButtonStyle.Primary,
  size = UanButtonSize.Regular,
  enabled = true,
  loading = false,
  fillWidth = false,
  leadingIcon = Icons.Default.Check,
  trailingIcon = null,
)

// Variante con slot de contenido libre
UanButton(onClick = { }) {
  Icon(Icons.Default.Warning, contentDescription = null)
  Text("Alerta")
}

// Estado de carga
var cargando by remember { mutableStateOf(false) }
UanButton(
  text = "Enviar",
  onClick = { cargando = true },
  loading = cargando,
)
```

UanCard



Figura 13. Superficie contenedora de información (UanCard)

UanCard agrupa información relacionada con jerarquía visual clara. Soporta tres estilos, contenido libre mediante slots, estado de carga y selección visual.

```
UanCard(
  title = "Protocolo activo",
  body = "Se ha activado el protocolo de evacuación del edificio A.",
  tone = UanTone.Warning,
  footer = {
    UanButton(text = "Ver detalles", onClick = { }, size = UanButtonSize.Compact)
  },
  supportingContent = {
    UanBadge(text = "Urgente", tone = UanTone.Danger)
  },
)
```

UanCarousel



Figura 14. Contenedor deslizable (UanCarousel)

UanCarousel presenta elementos de forma secuencial, como tarjetas de contenido destacado. Soporta avance automático configurable y respeta la preferencia `reduceMotion` del sistema operativo.

```
UanCarousel(  
  items = listaDeItems,  
  itemContent = { item ->  
    UanCard(title = item.titulo, body = item.descripcion)  
  },  
)
```

UanCheckbox

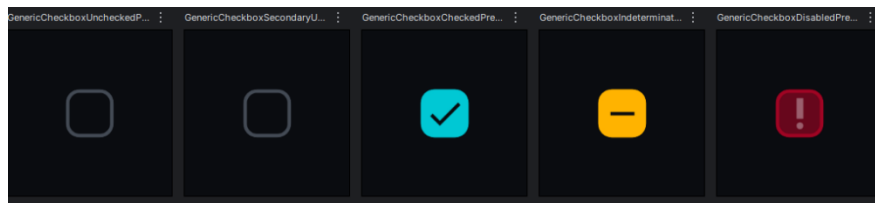


Figura 15. Control binario de selección (UanCheckbox)

UanCheckbox es un control binario de selección. Soporta tres estados (Unchecked, Checked, Indeterminate) y se controla externamente mediante el parámetro state.

```
var estado by remember { mutableStateOf(UanCheckboxState.Unchecked) }
```

```
UanCheckbox(  
    state = estado,  
    onStateChange = { estado = it },  
    tone = UanTone.Primary,  
    enabled = true,  
)
```

```
// Estados: Unchecked | Checked | Indeterminate
```

UanDivider

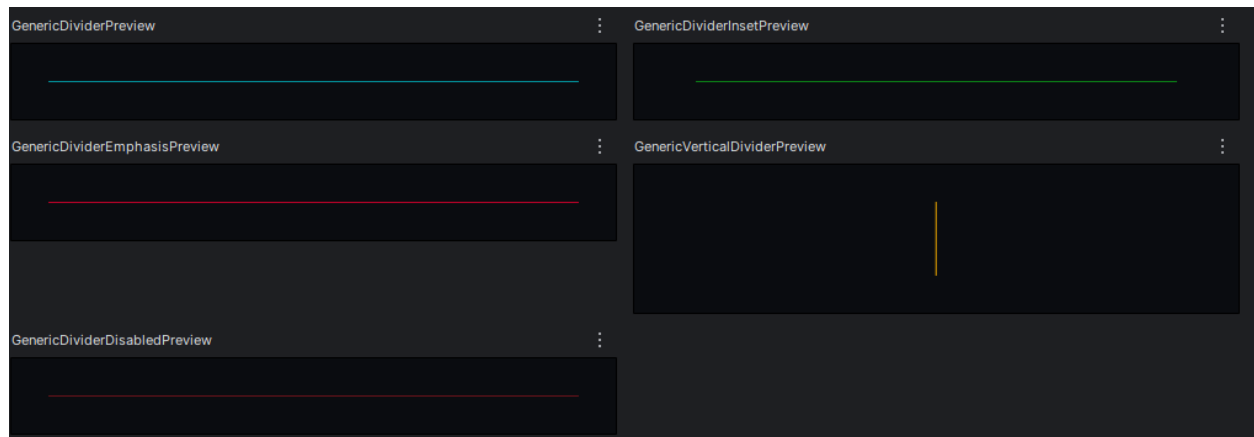


Figura 16. Separadores horizontal y vertical (UanDivider)

UanDivider separa bloques de contenido horizontalmente. UanVerticalDivider delimita columnas o grupos en fila. Ambos leen el token colors.outline del sistema.

```
// Horizontal
UanDivider(orientation = UanDividerOrientation.Horizontal)

// Vertical
UanDivider(orientation = UanDividerOrientation.Vertical)
```

UanDropdown

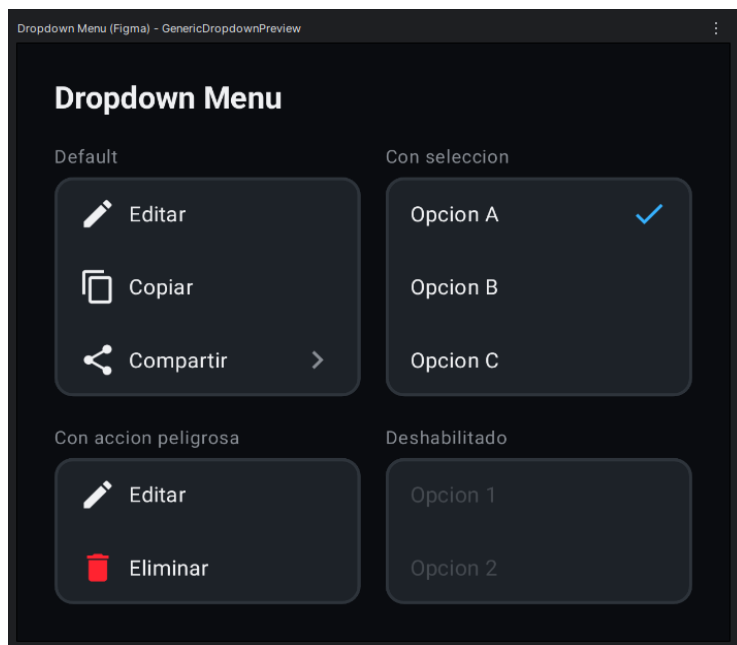


Figura 17. Menú contextual desplegable (UanDropdownMenu)

UanDropdown presenta un panel flotante con opciones de selección. Sigue el patrón de componentes controlados de Jetpack Compose: el estado expanded lo maneja el consumidor.

```
var seleccion by remember { mutableStateOf("") }

UanDropdown(
    label = "Selecciona una opción",
    selectedValue = seleccion,
    options = listOf("Opción A", "Opción B", "Opción C"),
    onOptionSelected = { seleccion = it },
)
```

UanElevation

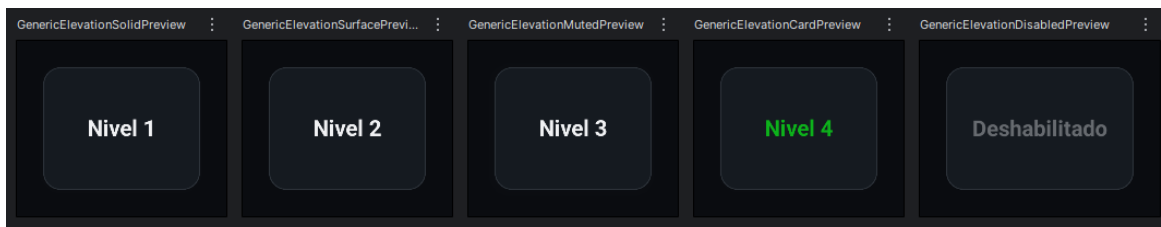


Figura 18. Profundidad visual (UanElevation)

UanElevation define los niveles de profundidad visual de los componentes mediante sombras. Se aplica a superficies con UanElevatedSurface para agregar jerarquía visual sin cambiar el color.

```
UanElevatedSurface(  
  elevation = UanElevation.Level2,  
  tone = UanTone.Neutral,  
) {  
  Text("Contenido con sombra")  
}
```

UanList

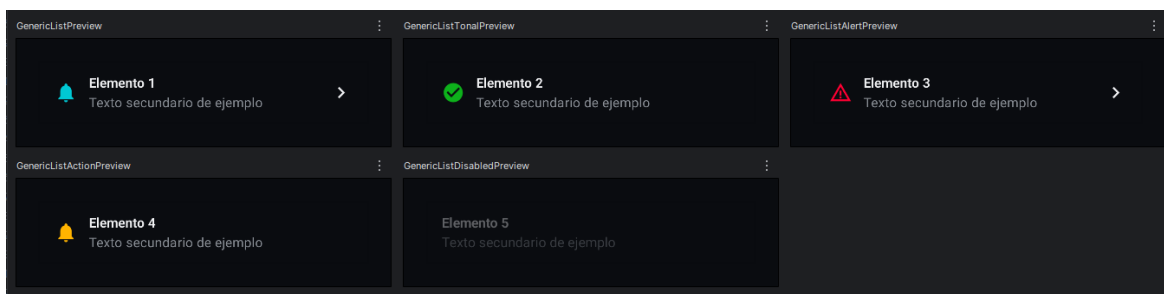


Figura 19. Lista estructurada de elementos (UanLists)

UanList construye listas optimizadas mediante LazyColumn con claves explícitas para minimizar recomposiciones. Los slots leading y trailing delimitan los casos de uso permitidos.

```
UanList(  
  items = listOf(  
    UanListItem(title = "Elemento 1", subtitle = "Descripción 1"),  
    UanListItem(title = "Elemento 2", subtitle = "Descripción 2"),  
  ),  
  onItemClick = { item -> /* acción */ },  
)
```

UanModal

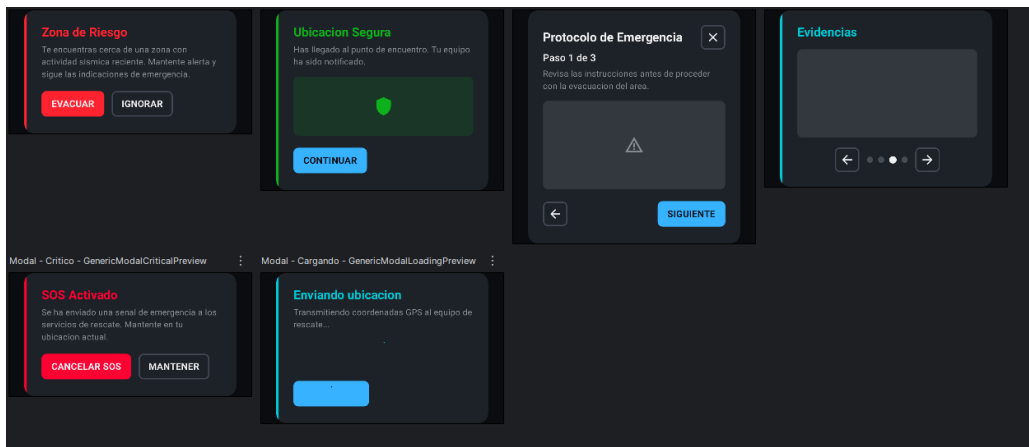


Figura 20. Diálogo emergente (UanModal)

UanModal (UanDialog) presenta mensajes importantes, confirmaciones y decisiones del usuario. Expone un slot composable body para contenido arbitrario y soporta hasta tres acciones simultáneas.

```
var visible by remember { mutableStateOf(false) }

UanModal(
    visible = visible,
    onDismissRequest = { visible = false },
    title = "Confirmar reporte",
    body = "¿Deseas enviar el reporte al personal de seguridad?",
    tone = UanTone.Danger,
    dismissOnClickOutside = false,
    primaryAction = UanModalAction(
        label = "Confirmar",
        onClick = { /* enviar */ visible = false },
        style = UanButtonStyle.Danger,
    ),
)
```

```
secondaryAction = UanModalAction(
    label = "Cancelar",
    onClick = { visible = false },
),
)
```

UanProgressIndicator

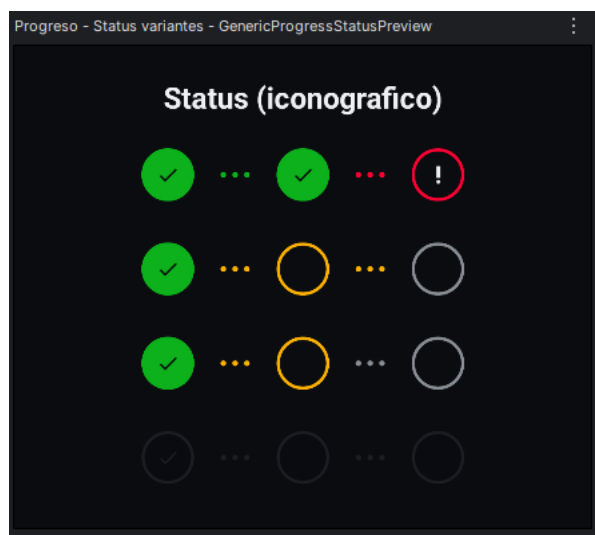


Figura 21. Indicador de progreso (UanProgressIndicator)

UanProgressIndicator comunica el avance de una operación al usuario. Implementa dos estilos (Circular y Linear) en tres tamaños (Small 16 dp, Medium 32 dp, Large 48 dp).

```
// Lineal con progreso determinado
UanProgressIndicator(
    progress = 0.65f,
    tone = UanTone.Primary,
)

// Circular indeterminado
UanProgressIndicator(
    indeterminate = true,
    tone = UanTone.Info,
```

)

UanNumberedProgress

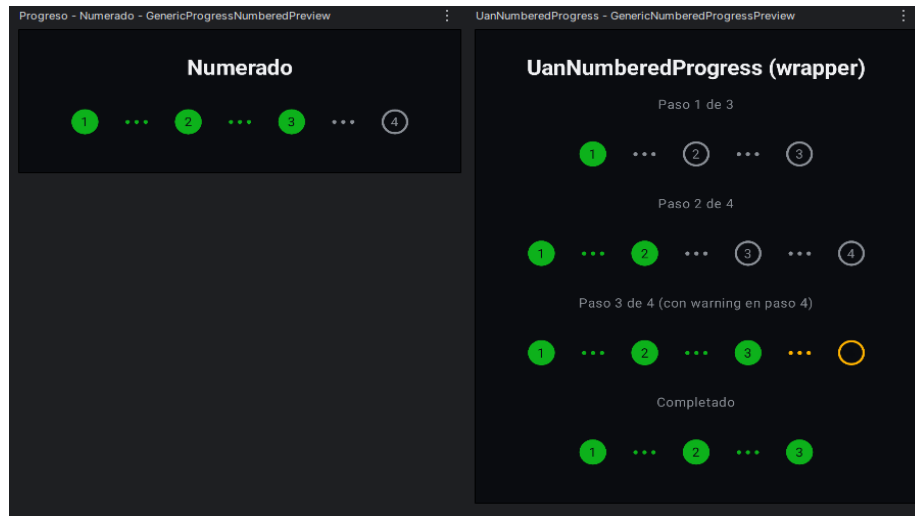


Figura 22. Progreso numerado por pasos (UanNumberedProgress)

UanNumberedProgress visualiza el progreso de un flujo de varios pasos. El paso actual se resalta visualmente y el componente soporta navegación lineal entre pasos.

```
UanNumberedProgress(  
  steps = 4,  
  currentStep = 2,  
  tone = UanTone.Primary,  
)
```

UanRadar



Figura 23. Visualización de rangos (UanRadar)

UanRadar renderiza una visualización de tipo radar para comparar niveles, rangos o métricas en múltiples ejes. Soporta nodos personalizables, animación de pulso y área táctil mínima de 48 dp por nodo.

```
UanRadar(  
  labels = listOf("Velocidad", "Potencia", "Resistencia", "Agilidad", "Precisión"),  
  values = listOf(0.8f, 0.6f, 0.9f, 0.5f, 0.7f),  
  tone = UanTone.Primary,  
)
```

UanSheet

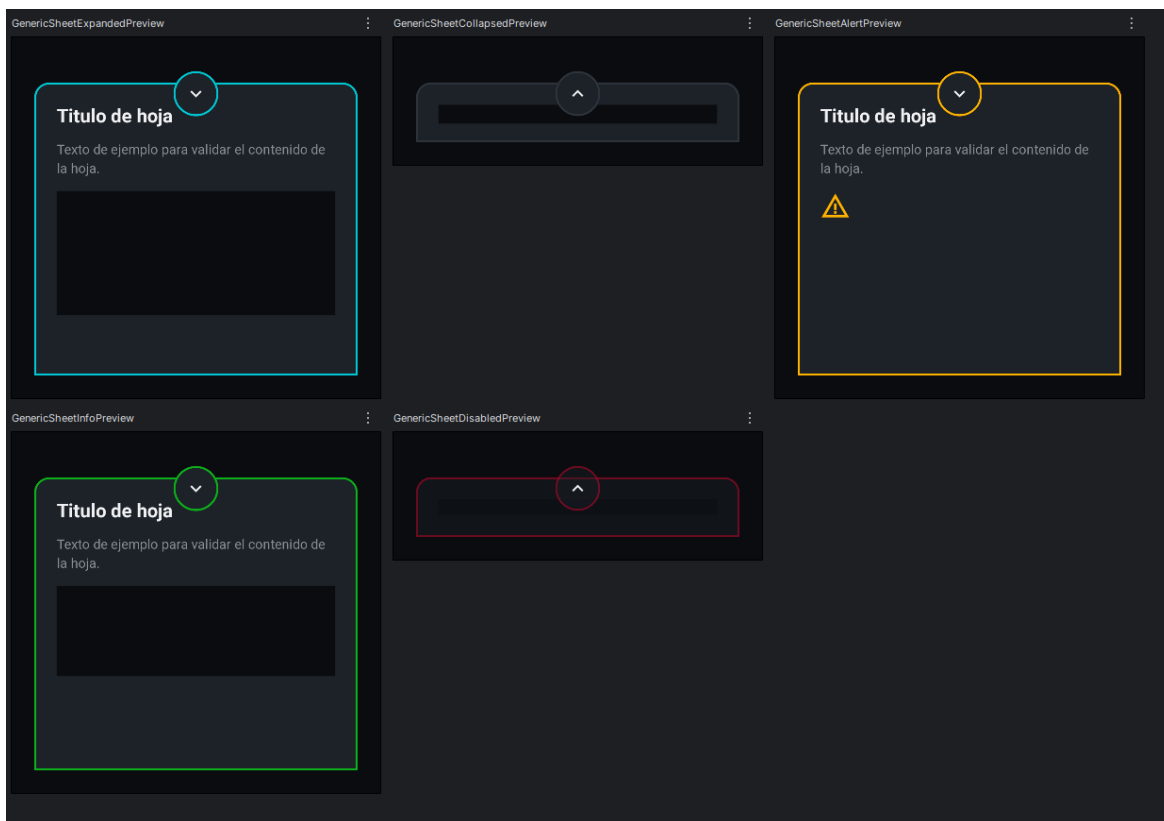


Figura 24. Panel inferior emergente (UanSheet)

UanSheet es un panel inferior emergente (bottom sheet) que presenta contenido adicional sin abandonar la pantalla actual. Soporta estados colapsado y expandido, altura configurable y contenido libre mediante slots.

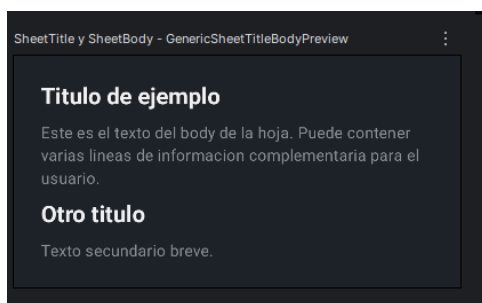


Figura 25. Título del panel inferior (UanSheetTitle)



Figura 26. Cuerpo del panel inferior (UanSheetBody)

```
var expandido by remember { mutableStateOf(false) }

UanSheet(
    expanded = expandido,
    onExpandedChange = { expandido = it },
    tone = UanTone.Warning,
    peekHeight = 80.dp,
    maxExpandedHeight = 300.dp,
    collapsedContent = {
        UanSheetTitle("Alerta activa")
    },
) {
    UanSheetTitle("Detalles de la alerta")
    UanSheetBody("Se detectó humo en el piso 3. Personal en camino.")
    UanButton(text = "Confirmar recepción", onClick = { })
}
```

UanSwitch

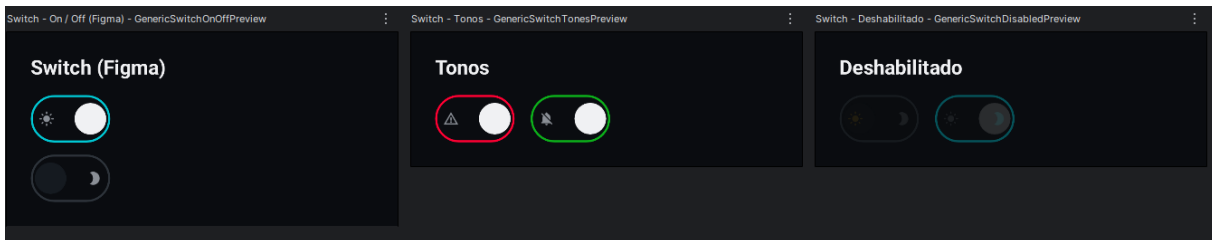


Figura 27. Control de encendido/apagado (UanSwitch)

UanSwitch es un control de encendido/apagado. Se controla externamente mediante el parámetro `checked` y notifica cambios mediante `onCheckedChange`.

```
var activado by remember { mutableStateOf(false) }
```

```
UanSwitch(  
    checked = activado,  
    onCheckedChange = { activado = it },  
    tone = UanTone.Primary,  
    enabled = true,  
)
```

UanTextField

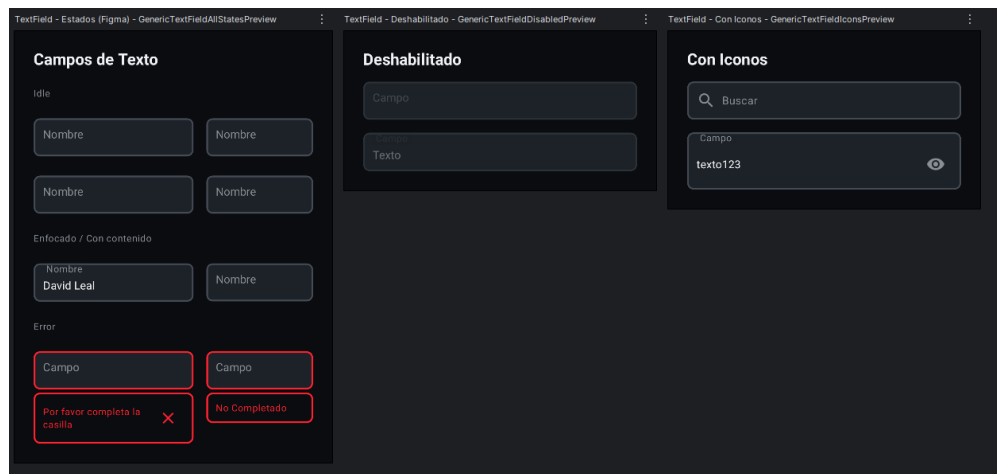


Figura 28. Campo de entrada de texto (UanTextField)

UanTextField es el componente estándar para capturar texto en formularios. Soporta decoradores, estado de error con texto de soporte y modo de solo lectura.

```
var texto by remember { mutableStateOf("") }

UanTextField(
    value = texto,
    onValueChange = { texto = it },
    label = "Nombre completo",
    placeholder = "Ej: Ana García",
    enabled = true,
    isError = false,
    errorMessage = null,
    leadingIcon = {
        Icon(Icons.Default.Person, contentDescription = null)
    },
    singleLine = true,
)

// Validación con mensaje de error
```

```
UanTextField(  
  value = email,  
  onValueChange = { email = it },  
  label = "Correo electrónico",  
  isError = !email.contains("@"),  
  errorMessage = "Ingresa un correo válido",  
)
```

UanToolbar



Figura 29. Barra de herramientas de acciones rápidas (UanToolbar)

UanToolbar agrupa acciones rápidas de navegación entre secciones. El elemento seleccionado se controla externamente mediante el parámetro `selectedItemId`. Asigna `Role.Tab` a cada elemento para que los lectores de pantalla interpreten correctamente la barra.

```
UanToolbar(  
  actions = listOf(  
    // ...
```

```

UanToolBarAction(
    icon = Icons.Default.Search,
    contentDescription = "Buscar",
    onClick = { },
),
UanToolBarAction(
    icon = Icons.Default.FilterList,
    contentDescription = "Filtrar",
    onClick = { },
),
),
)

```

2. Características de los Usuarios del Sistema

El Uan UI Kit está diseñado para ser consumido por dos perfiles de usuario técnico dentro del ecosistema de proyectos de investigación de la Facultad de Ingeniería de Sistemas:

Perfil	Descripción	Interacción con el sistema
Desarrollador Android	Estudiante o ingeniero que integra el UI Kit en una aplicación Android existente o nueva.	Declara el módulo :ui-kit como dependencia, llama composables desde su código, personaliza tokens mediante copy().

Diseñador / Investigador	Miembro del equipo que valida la consistencia visual y accesibilidad de los componentes.	Consulta la documentación de API, revisa los estados de cada componente, verifica criterios WCAG 2.2.
--------------------------	--	---

El sistema no está diseñado para usuarios finales de las aplicaciones móviles. Su consumidor directo es siempre un perfil técnico que integra la librería en sus propios proyectos Android.

3. Requisitos de Hardware y Software

3.1 Requisitos para desarrollo e integración

Requisito	Especificación
Sistema operativo	Windows 10/11, macOS 12+, o Linux Ubuntu 20.04+
Android Studio	Hedgehog (2023.1.1) o superior
JDK	Java 17 o superior
Kotlin	1.9.0 o superior
Android SDK mínimo	API 26 (Android 8.0 Oreo)
Android SDK objetivo	API 35 (Android 15)

Compose BOM	2024.09.00
RAM mínima	8 GB (16 GB recomendado)
Almacenamiento	10 GB libres para Android SDK y dependencias

3.2 Requisitos para dispositivos de prueba

Requisito	Especificación
Android OS	Android 8.0 (API 26) o superior
Tamaño de pantalla	4.7" o superior
Accesibilidad	TalkBack habilitado para pruebas de accesibilidad
Emulador compatible	AVD con API 26+ en Android Studio

4. Instrucciones de Instalación e Integración

Requisitos previos

- Android Studio Hedgehog o superior
- minSdk 26 o superior
- Kotlin con Jetpack Compose habilitado

Paso 1 — Agregar el repositorio Maven

En tu archivo settings.gradle.kts:

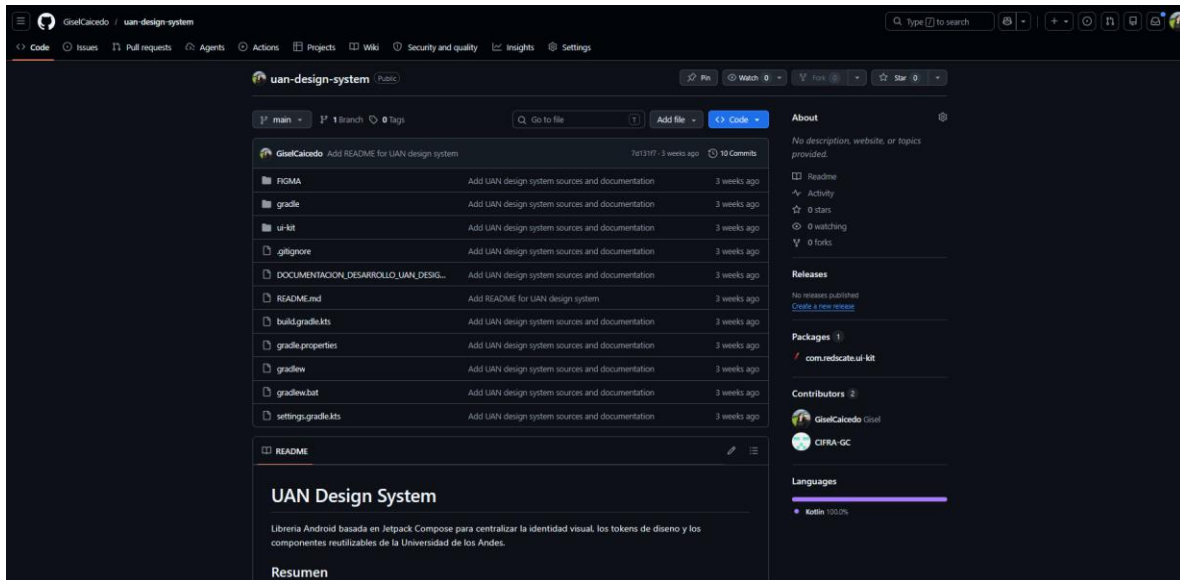
```
dependencyResolutionManagement {  
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)  
    repositories {  
        google()  
        mavenCentral()  
        maven {  
            url = uri("https://maven.pkg.github.com/GiselCaicedo/uan-ui-kit")  
            credentials {  
                username = providers.gradleProperty("gpr.user").orNull  
                    ?: System.getenv("GITHUB_USERNAME")  
                password = providers.gradleProperty("gpr.token").orNull  
                    ?: System.getenv("GITHUB_TOKEN")  
            }  
        }  
    }  
}
```

Paso 2 — Agregar la dependencia

En el build.gradle.kts de tu módulo de app:

```
dependencies {  
    implementation("co.edu.uan.ingsis:ui-kit:1.0.0")  
}
```

Paso 3 — Sincronizar el proyecto



Paso 3 — Sincronizar el proyecto

Haz clic en «Sync Now» en Android Studio después de agregar la dependencia, o ejecuta desde la terminal:

```
./gradlew :app:dependencies --configuration debugRuntimeClasspath
```

Resultado esperado: el artefacto `co.edu.uan.ingsis:ui-kit:1.0.0` debe aparecer en el árbol de dependencias sin errores de resolución. Si la descarga fue exitosa verás una línea similar a:

```
co.edu.uan.ingsis:ui-kit:1.0.0 -> 1.0.0 (*)
```

Posibles errores:

- 401 Unauthorized: token de GitHub Packages inválido o vencido. Genera un nuevo token con permiso `read:packages`.
- Could not resolve `co.edu.uan.ingsis:ui-kit:1.0.0`: la URL del repositorio es incorrecta. Verifica

que uses <https://maven.pkg.github.com/GiselCaicedo/uan-ui-kit>

- Unsupported class file major version: la versión de JDK no es compatible. Usa JDK 17 o superior.

```

terminal Local +
+--- androidx.compose.material:material:1.8.1 (c)
+--- androidx.compose.ui.tooling:ui-tooling-data:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.animation:animation:android:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.ui.tooling:ui-tooling-data:android:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.material:material:android:1.8.1 (c)
+--- androidx.compose.animation:animation-core:android:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.foundation:foundation-layout:android:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.foundation:foundation:android:1.8.1 -> 1.9.2 (c)
+--- androidx.compose.ui:ui -> 1.9.2 (*)
+--- androidx.compose.ui:ui-graphics -> 1.9.2 (*)
+--- androidx.compose.ui:ui-tooling-preview -> 1.9.2 (*)
+--- androidx.compose.material3:material3 -> 1.3.2
+--- androidx.compose.material3:material3:android:1.3.2
+--- androidx.activity:activity-compose:1.8.2 -> 1.10.0 (*)
+--- androidx.annotation:annotation:1.1.0 -> 1.9.1 (*)
+--- androidx.annotation:annotation-experimental:1.4.0 -> 1.4.1 (*)
+--- androidx.collection:collection:1.4.0 -> 1.5.0 (*)
+--- androidx.compose.animation:animation-core:1.6.0 -> 1.9.2 (*)
+--- androidx.compose.foundation:foundation:1.7.0 -> 1.9.2 (*)
+--- androidx.compose.foundation:foundation-layout:1.7.0 -> 1.9.2 (*)
+--- androidx.compose.material:material-icons-core:1.6.0 -> 1.7.8
+--- androidx.compose.material:material-icons-core:android:1.7.8
+--- androidx.compose.ui:ui:1.6.0 -> 1.9.2 (*)
+--- androidx.compose.ui:ui:1.6.0 -> 1.9.2 (*)
+--- org.jetbrains.kotlin:kotlin-stdlib:1.8.22 -> 2.2.10 (*)
+--- org.jetbrains.kotlin:kotlin-stdlib-common:1.8.22 -> 2.2.10 (*)
+--- androidx.compose.material:material-ripple:1.7.0 -> 1.8.1 (*)
+--- androidx.compose.runtime:runtime:1.7.0 -> 1.9.2 (*)
+--- androidx.compose.ui:ui:1.6.0 -> 1.9.2 (*)
+--- androidx.compose.ui:ui:1.6.0 -> 1.9.2 (*)
+--- androidx.compose.ui:ui-util:1.6.0 -> 1.9.2 (*)
+--- androidx.lifecycle:lifecycle-common:java22:2.8.1 -> 2.10.0
+--- androidx.annotation:annotation:1.8.1 -> 1.9.1 (*)
+--- androidx.lifecycle:lifecycle-common:2.10.0 (*)
+--- androidx.lifecycle:lifecycle-common:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-livedata-core:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-livedata:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-runtime:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-runtime-compose:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-runtime-rtcr:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-viewmodel:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-viewmodel-savedstate:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-viewmodel-std:2.10.0 (c)
+--- androidx.lifecycle:lifecycle-process:2.10.0 (c)
+--- org.jetbrains.kotlin:kotlin-stdlib-common:1.8.22 -> 2.2.10 (*)
+--- androidx.compose.material:material-icons-extended -> 1.7.8
+--- androidx.compose.material:material-icons-extended:android:1.7.8
+--- org.jetbrains.kotlin:kotlin-stdlib:1.8.22 -> 2.2.10 (*)
+--- com.uan.designsystem:uan-design-system:1.0.0
+--- androidx.compose.compose:compose-bom:2024.09.00 -> 2025.05.00 (*)
+--- androidx.compose.ui:ui -> 1.9.2 (*)
+--- androidx.compose.foundation:foundation -> 1.9.2 (*)
+--- androidx.compose.material3:material3 -> 1.3.2 (*)
+--- androidx.compose.material:material-icons-extended -> 1.7.8 (*)
+--- androidx.compose.ui:ui-tooling-preview -> 1.9.2 (*)
+--- androidx.activity:activity-compose:1.9.0 -> 1.11.0 (*)
+--- org.jetbrains.kotlin:kotlin-stdlib:2.2.10 (*)

(c) - A dependency constraint, not a dependency. The dependency affected by the constraint occurs elsewhere in the tree.
(*) - Indicates repeated occurrences of a transitive dependency subtree. Gradle expands transitive dependency subtrees only once per project; repeat occurrences only display the root of the subtree, followed by this annotation.
A web-based, searchable dependency report is available by adding the --scan option.

BUILD SUCCESSFUL in 11ms
1 actionable task: 1 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.1/userguide/configuration_cache_enabling.html
PS C:\Users\gsan\AndroidStudioProjects\prueba\uan>
  
```

Figura. Árbol de dependencias con co.edu.uan.ingsis:ui-kit:1.0.0 resuelto. Fuente: elaboración propia.

Paso 4 — Envolver tu UI con UanTheme

```
import com.uan.designsystem.uikit.theme.UanTheme
```

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            UanTheme {
                // Tu contenido aquí
            }
        }
    }
}
  
```

Paso 5 — Usar los tokens del tema

```
@Composable
fun MiPantalla() {
    UanTheme {
        val tokens = UanThemeTokens.current
        Text(
            text = "Hola UAN",
            style = tokens.typography.titleLarge,
            color = tokens.colors.primary
        )
    }
}
```

Paso 6 — Configurar credenciales de GitHub (opcional)

Si el paquete es privado, agrega tus credenciales en ~/.gradle/gradle.properties:

```
gpr.user=TU_USUARIO_GITHUB
gpr.token=TU_PERSONAL_ACCESS_TOKEN
```

El token requiere el permiso read:packages. Para generarlo: GitHub › Settings › Developer settings › Personal access tokens › Generate new token (classic) › seleccionar read:packages.

⚠ ADVERTENCIA DE SEGURIDAD — Manejo seguro de credenciales:

Los tokens de acceso a GitHub Packages son credenciales sensibles. Nunca deben incluirse directamente en archivos de código fuente ni en archivos versionados con Git. El incumplimiento

de esta norma expone el token públicamente y puede comprometer el repositorio.

Prácticas obligatorias:

- Almacenar el token en ~/.gradle/gradle.properties (archivo local, NUNCA commitear).
- Agregar gradle.properties al .gitignore del proyecto si no está ya incluido.
- En entornos de CI/CD (p. ej. GitHub Actions), usar variables de entorno del repositorio:

Settings › Secrets and variables › Actions › New repository secret.

- Rotar el token periódicamente o cuando se sospeche de exposición.
- Si el token fue expuesto accidentalmente (commit público), revocarlo de inmediato en GitHub

› Settings › Developer settings › Personal access tokens.

Repositorio oficial: github.com/GiselCaicedo/uan-ui-kit

Validación de compilación completa:

Una vez completados los pasos anteriores, ejecuta la siguiente secuencia para verificar la integración:

```
# Compilar el módulo de app incluyendo el Uan UI Kit
```

```
./gradlew :app:compileDebugKotlin
```

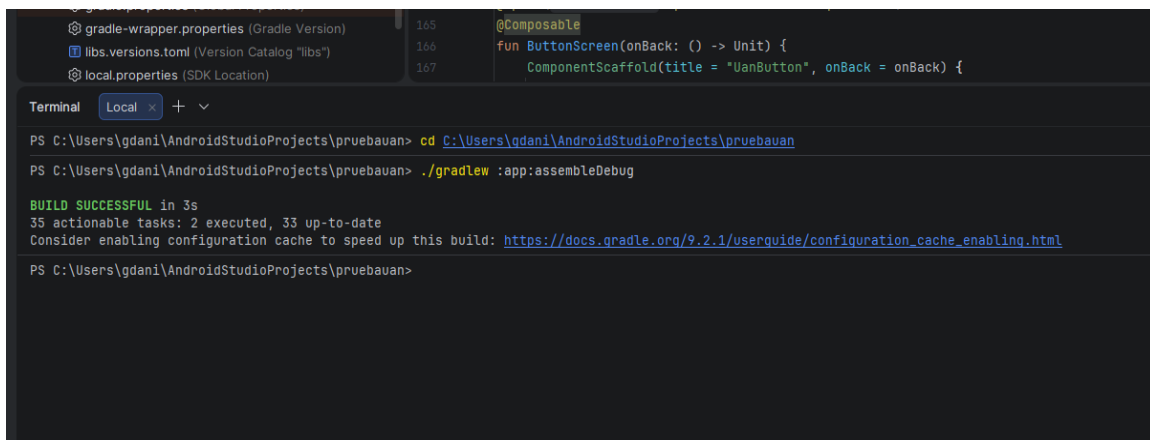
```
# Ejecutar las pruebas unitarias del módulo ui-kit
```

```
./gradlew :ui-kit:test
```

Generar el APK de debug

`./gradlew :app:assembleDebug`

Resultado esperado: BUILD SUCCESSFUL en las tres instrucciones. El archivo app-debug.apk se generará en `app/build/outputs/apk/debug/`.



```

@Composable
fun ButtonScreen(onBack: () -> Unit) {
    ComponentScaffold(title = "UanButton", onBack = onBack) {

```

```

Terminal Local x + v
PS C:\Users\gdani\AndroidStudioProjects\pruebauan> cd C:\Users\gdani\AndroidStudioProjects\pruebauan
PS C:\Users\gdani\AndroidStudioProjects\pruebauan> ./gradlew :app:assembleDebug

BUILD SUCCESSFUL in 3s
35 actionable tasks: 2 executed, 33 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.1/userguide/configuration_cache_enabling.html
PS C:\Users\gdani\AndroidStudioProjects\pruebauan>

```

Figura. Resultado de `./gradlew :app:assembleDebug` — BUILD SUCCESSFUL. Fuente: elaboración propia.



```

local.properties (SDK Location) 167 ComponentScaffold(title = "UanButton", onBack = onBack) {

```

```

Terminal Local x + v
PS C:\Users\gdani\AndroidStudioProjects\UANDSIGNSYSTEM> ./gradlew :ui-kit:test

BUILD SUCCESSFUL in 17s
12 actionable tasks: 7 executed, 5 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.1/userguide/configuration_cache_enabling.html
PS C:\Users\gdani\AndroidStudioProjects\UANDSIGNSYSTEM> 

```

Figura. Resultado de `./gradlew :ui-kit:test` — BUILD SUCCESSFUL. Fuente: elaboración propia.

4.1 Configuración del repositorio GitHub Packages

El módulo `:ui-kit` se distribuye como artefacto `.aar` publicado en GitHub Packages. Para consumirlo, es necesario configurar las credenciales de acceso en el proyecto Android:

Paso 1: Agregar el repositorio en settings.gradle.kts

```
dependencyResolutionManagement {  
  
    repositories {  
  
        maven {  
  
            url = uri("https://maven.pkg.github.com/GiselCaicedo/uan-ui-kit")  
  
            credentials {  
  
                username = providers.gradleProperty("gpr.user").orNull  
                ?: System.getenv("GITHUB_USERNAME")  
  
                password = providers.gradleProperty("gpr.token").orNull  
                ?: System.getenv("GITHUB_TOKEN")  
  
            }  
  
        }  
  
    }  
  
}
```

Paso 2: Agregar la dependencia en build.gradle.kts del módulo consumidor

```
dependencies {  
  
    implementation("co.edu.uan.ingsis:ui-kit:1.0.0")  
  
}
```

Paso 3: Agregar credenciales en local.properties (no commitar)

```
gpr.user=TU_USUARIO_GITHUB  
  
gpr.token=TU_PERSONAL_ACCESS_TOKEN
```

4.2 Configuración del tema en la aplicación

El módulo consumidor debe envolver su árbol de composición con el composable UanTheme en la raíz de la actividad:

```
class MainActivity : ComponentActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
  
        super.onCreate(savedInstanceState)  
  
        setContent {  
  
            UanTheme {  
  
                // Contenido de la aplicación  
  
            }  
  
        }  
  
    }  
}
```

4.3 Personalización de tokens

Para adaptar la identidad visual sin modificar el código fuente de la librería, se utiliza la función copy() de Kotlin sobre UanTokens:

```
val misToknes = UanTokens.Default.copy(
```

```

        colors = UanTokens.Default.colors.copy(

        primary = Color(0xFF0057B8)

        )

    )

    UanTheme(tokens = misToknes) {

        // Árbol de composición con identidad visual personalizada

    }

```

4.4 Dependencias externas del módulo :ui-kit

Dependencia	Propósito
compose-bom:2024.09.00	Gestiona las versiones de todas las librerías de Compose
androidx.compose.ui:ui	Primitivas base: Modifier, Canvas, Layout
androidx.compose.foundation:foundation	LazyColumn, HorizontalPager, gestos e interacciones
androidx.compose.material3:material3	Tokens semánticos de Material You y componentes de superficie
material-icons-extended	Conjunto ampliado de iconos vectoriales Icons.Filled.*

androidx.activity:activity-compose:1.9.0	Integración de Compose con ComponentActivity
--	---

5. Solución de Problemas

5.1 Monitoreo y verificación del sistema

5.1.1 Verificación de integración exitosa

Para confirmar que el módulo está correctamente integrado, se puede ejecutar la siguiente instrucción de compilación desde Android Studio o la terminal:

```
./gradlew :app:compileDebugKotlin
```

Una compilación exitosa sin errores relacionados con co.edu.uan.ingsis:ui-kit confirma la integración correcta.

5.1.2 Pruebas unitarias del módulo

El Uan UI Kit incluye 38 pruebas unitarias distribuidas de la siguiente forma:

- 16 pruebas de estado de componente: verifican que cada componente renderiza correctamente los estados habilitado, deshabilitado y semánticos (error, cargando, seleccionado).
- 12 pruebas de tokens: validan que los valores de UanColors, UanTypography, UanShapes y UanMotion son propagados correctamente a través del CompositionLocal de UanTheme.
- 10 pruebas de accesibilidad estructural: verifican que los parámetros contentDescription, Role y semantics están presentes en los composables interactivos.

Comando de ejecución:

```
./gradlew :ui-kit:test
```

Resultado esperado:

BUILD SUCCESSFUL

38 tests completed, 0 failures

Los reportes HTML de las pruebas se generan en:

ui-kit/build/reports/tests/test/index.html

Pruebas funcionales (integración):

```
./gradlew :app:connectedDebugAndroidTest
```

Requiere un dispositivo o emulador Android conectado. Ejecuta pruebas de UI con Espresso verificando la composición visual de los componentes en pantalla real.

Pruebas de compatibilidad:

El sistema fue validado en Android API 26 (minSdk), API 30 y API 34. Para verificar en un emulador específico:

```
./gradlew :app:assembleDebug && adb install app/build/outputs/apk/debug/app-debug.apk
```

```
./gradlew :ui-kit:test
```

Todos los tests deben pasar en verde. Si alguno falla, se debe revisar la compatibilidad de la versión del BOM de Compose.

5.1.3 Verificación de accesibilidad con TalkBack

Para verificar el comportamiento de accesibilidad de los componentes interactivos del Uan UI Kit se utilizan dos herramientas complementarias:

A) TalkBack (verificación dinámica):

1. Habilita TalkBack: Ajustes › Accesibilidad › TalkBack › Activar.
2. Abre el módulo :app en el dispositivo de prueba.
3. Navega con el gesto de deslizamiento lateral (swipe derecha/izquierda) para recorrer los elementos interactivos en orden de foco.
4. Verifica que TalkBack anuncia: rol del componente (botón, casilla, interruptor), estado actual (activado, desactivado, seleccionado) y la descripción de contenido (contentDescription configurada en el componente).
5. Para componentes con ícono, confirma que la descripción no es vacía ni genérica (p. ej. 'imagen' no es aceptable; 'Volver a la pantalla anterior' sí lo es).

B) Accessibility Scanner (verificación automatizada):

1. Instala Accessibility Scanner desde Google Play.
2. Habilita la herramienta desde Ajustes › Accesibilidad › Accessibility Scanner.
3. Abre cada pantalla del módulo :app y pulsa el botón de escaneo (ícono de lupa azul).
4. Revisa el informe generado: cada hallazgo indica el componente afectado, el criterio

incumplido (contraste, área táctil, etiqueta faltante) y una recomendación de corrección.

5. Exporta el informe como HTML desde Acceso a informes dentro de la app.

Criterios de aceptación verificados:

- WCAG 2.2 criterio 1.4.3: relación de contraste texto/fondo $\geq 4.5:1$
- WCAG 2.2 criterio 1.4.11: contraste de componentes no textuales $\geq 3:1$
- WCAG 2.2 criterio 2.5.5: área táctil mínima 48×48 dp
 - Habilitar TalkBack en el dispositivo de prueba: Ajustes > Accesibilidad > TalkBack.
 - Navegar por los componentes implementados con el gesto de deslizamiento lateral.
 - Verificar que cada elemento interactivo recibe foco y TalkBack anuncia el rol y estado correctamente.
 - Utilizar Accessibility Scanner de Android para obtener un reporte automatizado de incumplimientos.

5.2 Problemas comunes y soluciones

Síntoma	Causa probable	Solución
Error 401 al descargar dependencia	Credenciales de GitHub Packages incorrectas o expiradas	Generar un nuevo Personal Access Token (PAT) con permisos read:packages y actualizar local.properties

UanTheme no encontrado al compilar	Módulo :ui-kit no declarado como dependencia	Verificar la declaración en build.gradle.kts del módulo consumidor
Componente no respeta los tokens de color	UanTheme no declarado en la raíz del árbol de composición	Envolver el contenido con UanTheme en la actividad principal
TalkBack no anuncia el componente	Componente carece de bloque semantics o contentDescription	Verificar que el componente tenga el parámetro contentDescription asignado
Error de compatibilidad de BOM	Versión de Compose BOM incompatible con la versión de Kotlin	Usar exactamente compose-bom:2024.09.00 con Kotlin 1.9.0+
Animaciones no se detienen con reduceMotion	Preferencia del sistema no leída correctamente	Verificar que el composible utiliza LocalContext o la API de Compose para leer la preferencia

5.3 Pasos para reinstalación limpia del módulo

En caso de corrupción del caché de Gradle o problemas de sincronización, ejecutar los siguientes pasos en orden:

1. Limpiar el caché de Gradle:

```
./gradlew clean
```

2. Invalidar caches de Android Studio:

File > Invalidate Caches > Invalidate and Restart

3. Forzar la descarga de dependencias:

```
./gradlew --refresh-dependencies :app:assembleDebug
```

5.4 Contacto de soporte técnico

Contacto	Información
Desarrolladora	Gisel Daniela Caicedo Soler
Correo electrónico	giselcaicedosoler@gmail.com
Director del proyecto	Ph.D. Elio Higinio Cables Pérez
Facultad	Ingeniería de Sistemas — UAN
Repositorio GitHub	https://github.com/GiselCaicedo/uan-ui-kit